CSC1109 – Data (Analytics) at Speed and Scale

Alessandra Mileo

alessandra.mileo@dcu.ie

Focus for Today

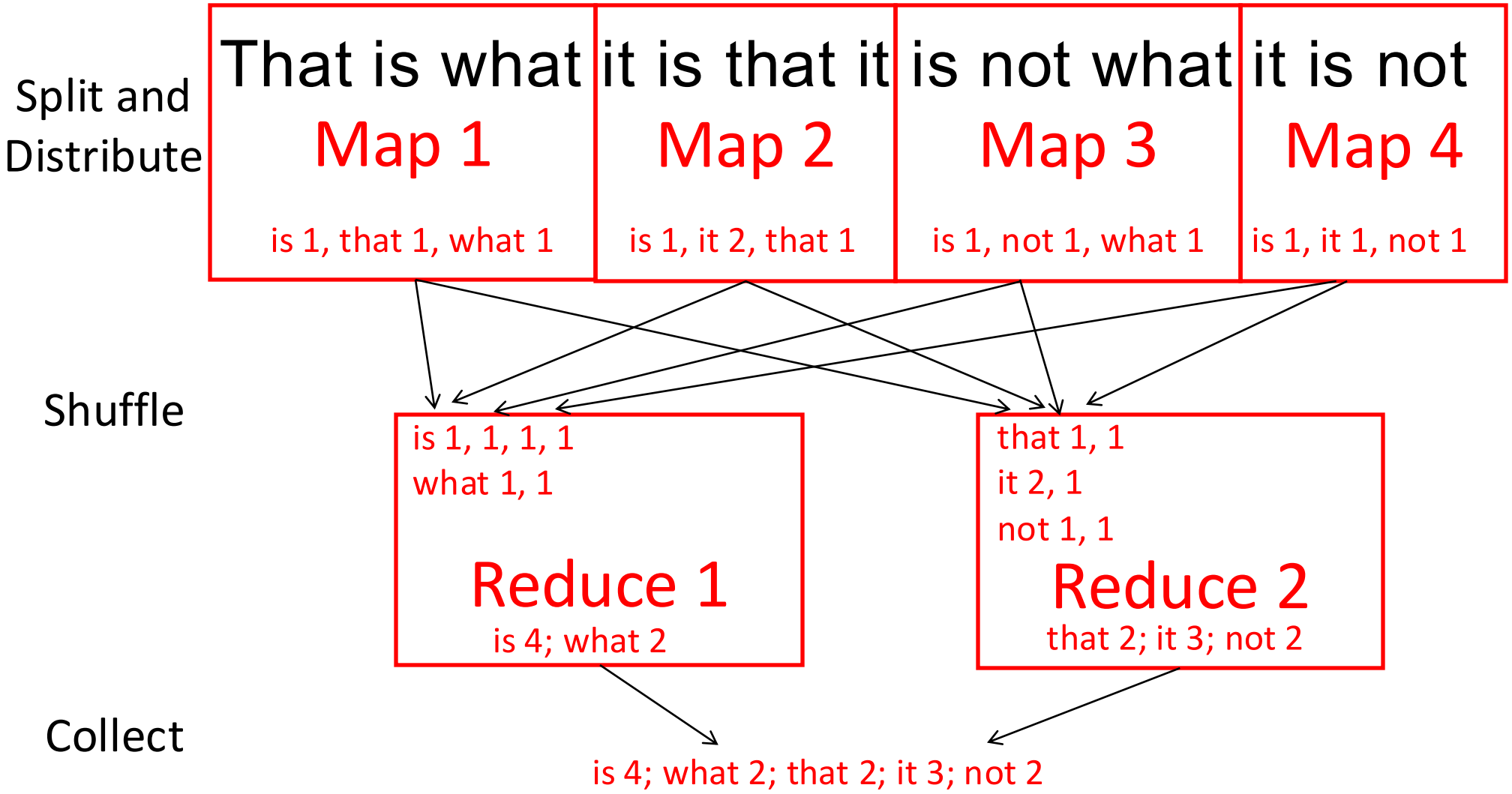
- MR quick recap
  - Phases of Execution
  - Combiner optimisation
- Another MR example
- Hadoop Cluster
  - Key components
  - Deploying a Hadoop Cluster (Lab1)
- Run WordCount example (from example code)
  - Java
  - Python

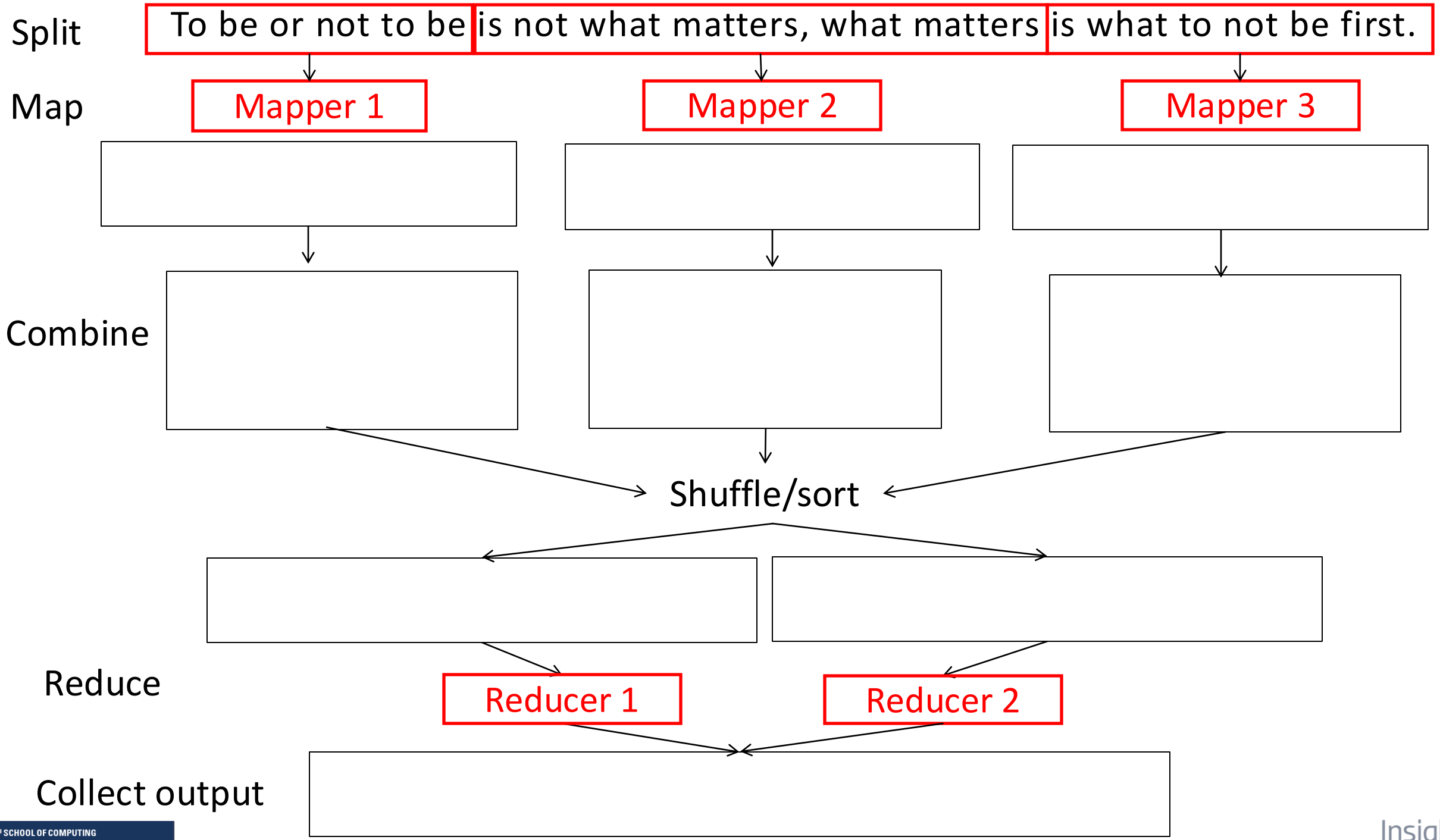
Focus for Today

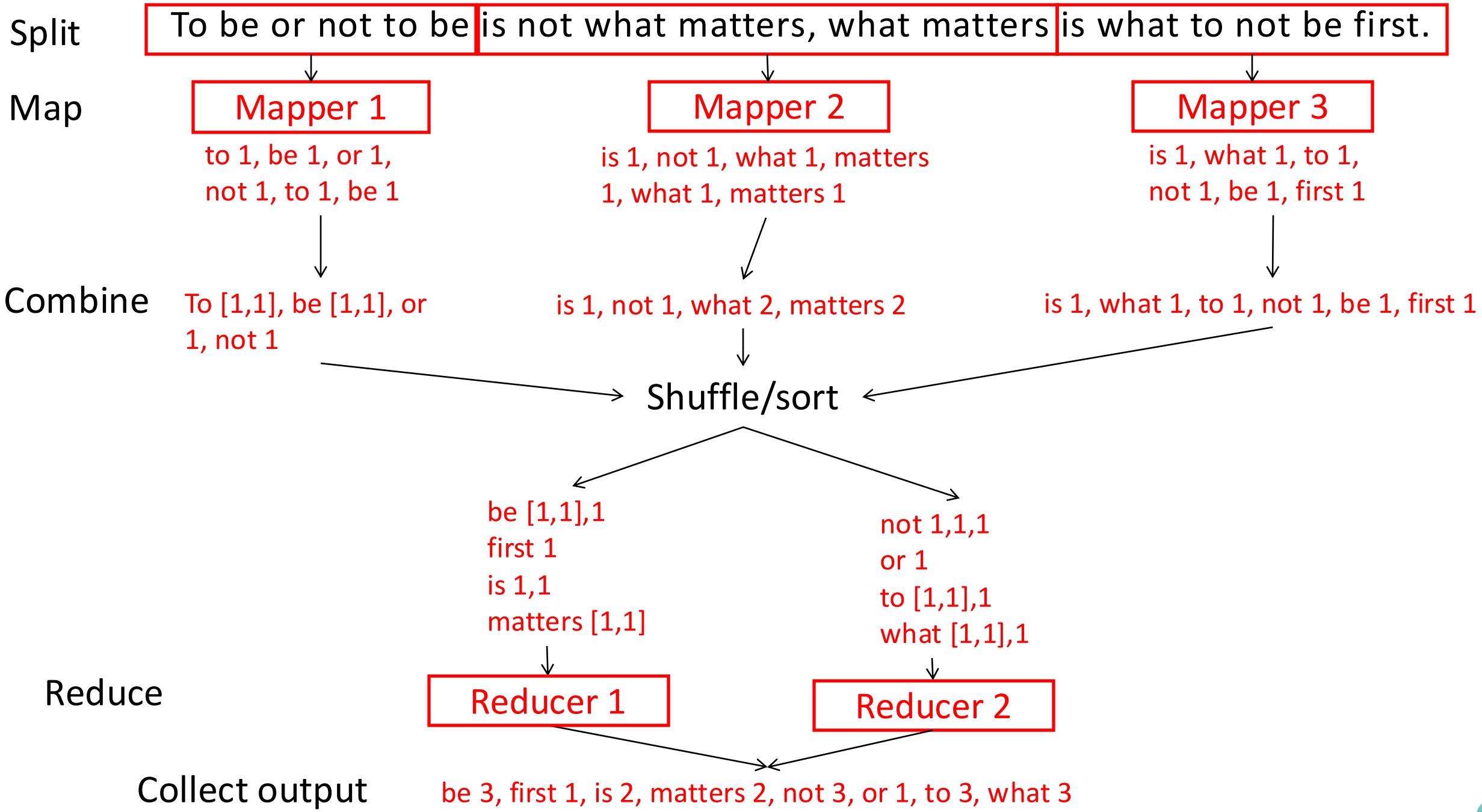
- MR quick recap
  - Phases of Execution
  - Combiner optimisation
- Another MR example
- Hadoop Cluster
  - Key components
  - Deploying a Hadoop Cluster (Lab1)
- Run WordCount example (from example code)
  - Java
  - Python

MapReduce Phases of Execution

Wordcount Example

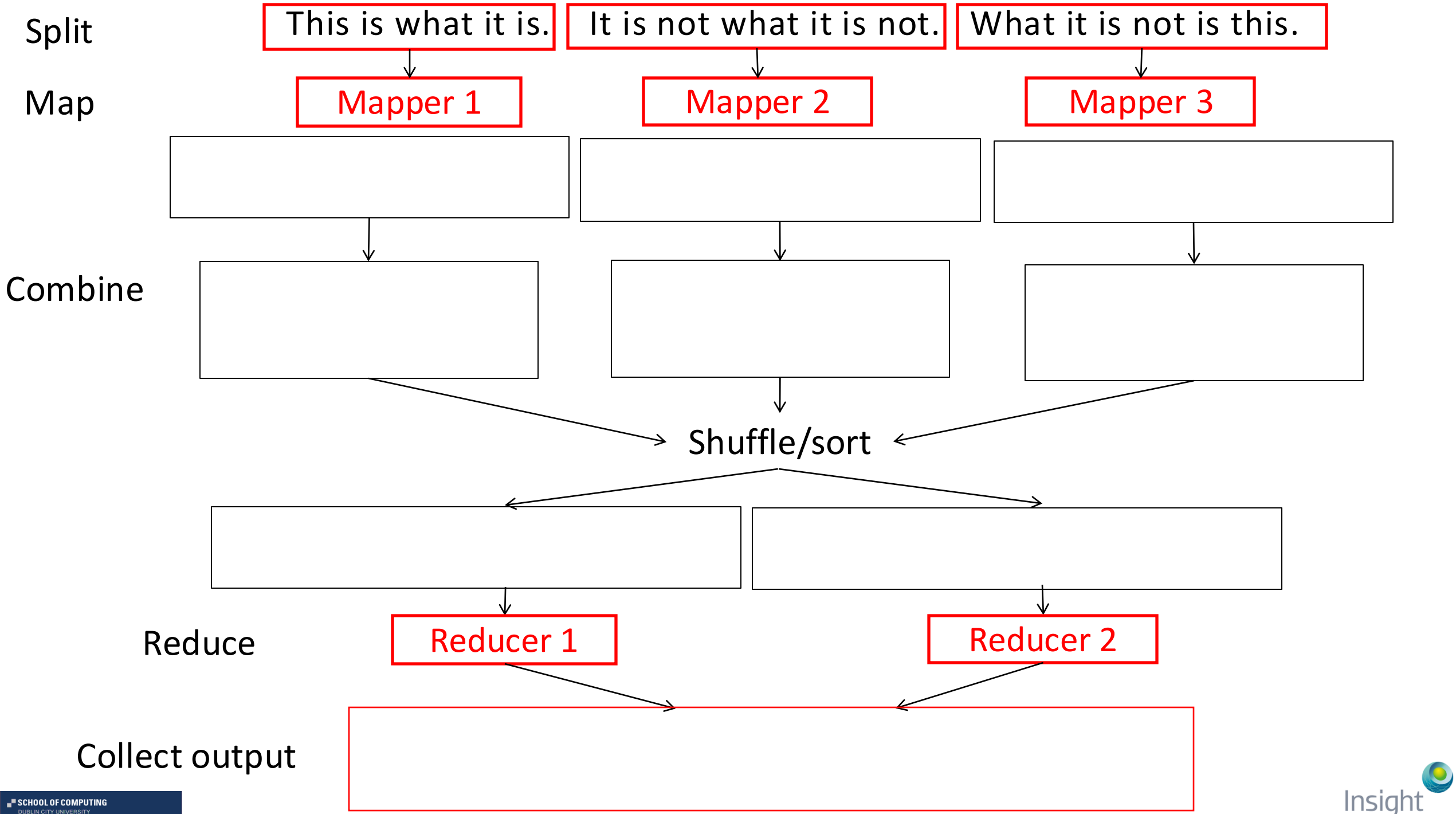


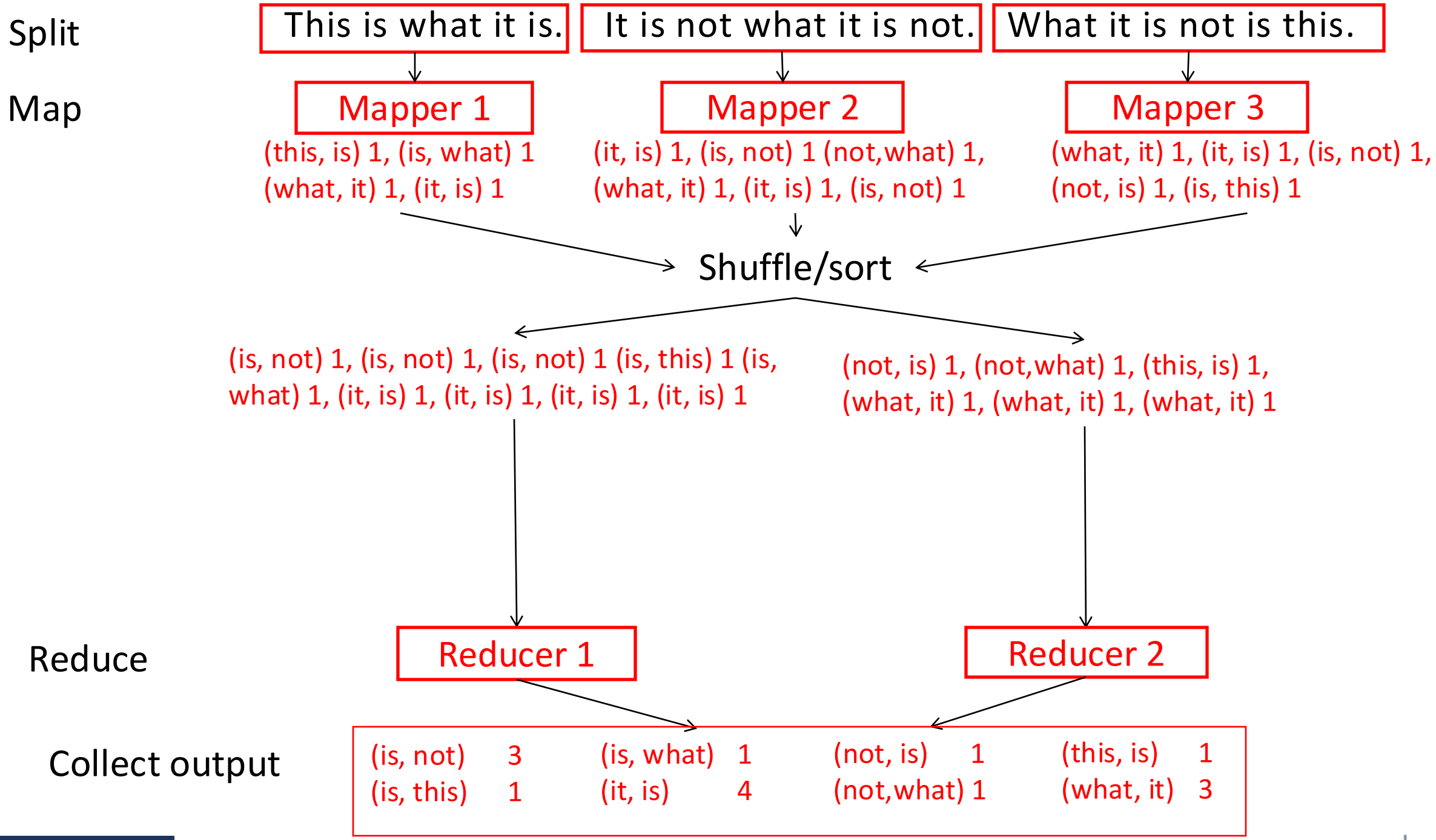

Split
To be or not to be
is not what matters, what matters
is what to not be first.
Map
Mapper 1
Mapper 2
Mapper 3
Combine
Shuffle/sort
Reduce
Reducer 1
Reducer 2
Collect output

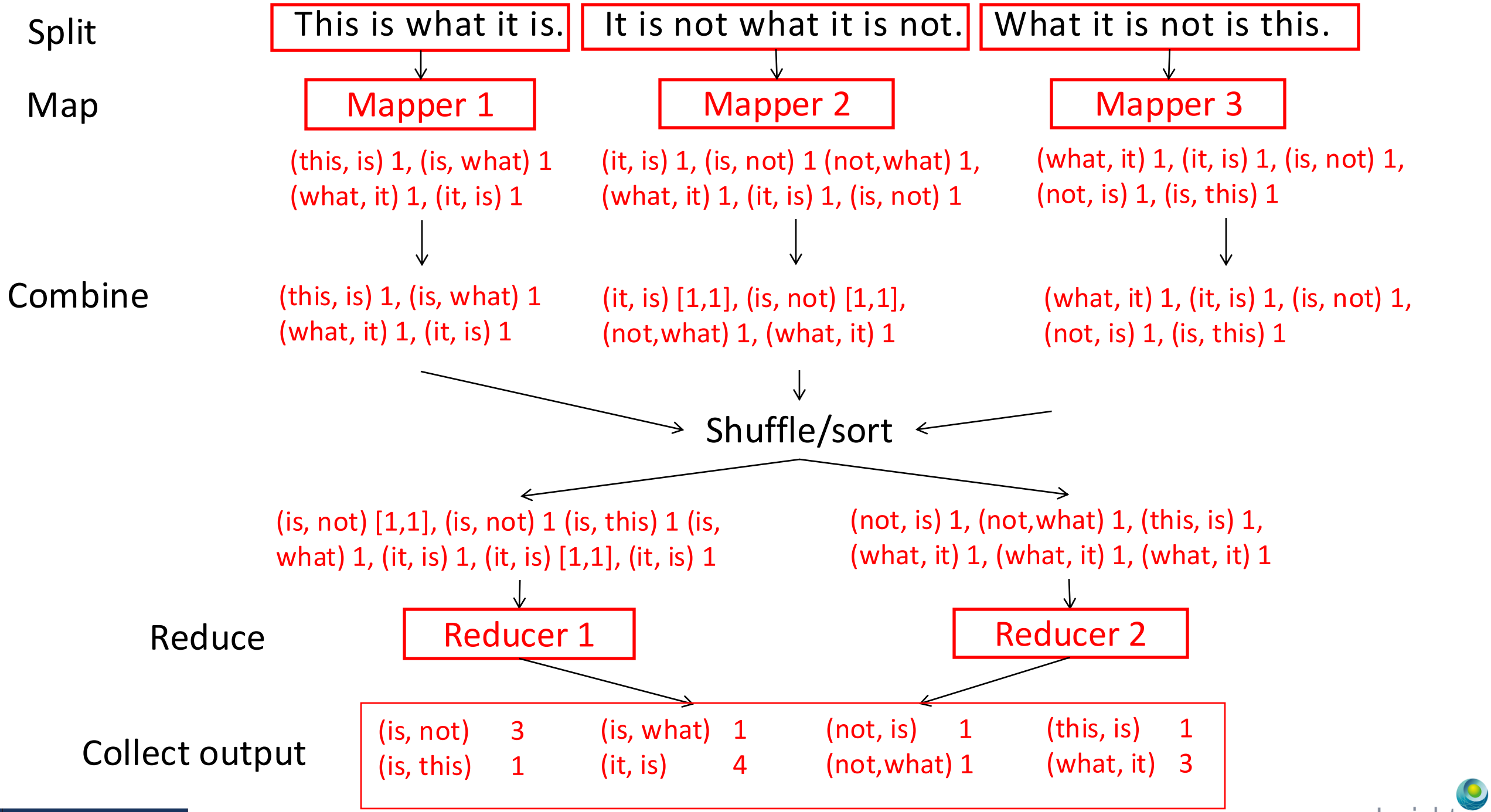

Split
To be or not to be
is not what matters, what matters
is what to not be first.
Map
Mapper 1
Mapper 2
Mapper 3
to 1, be 1, or 1,
not 1, to 1, be 1
is 1, not 1, what 1, matters
1, what 1, matters 1
is 1, what 1, to 1,
not 1, be 1, first 1
Combine
To [1,1], be [1,1], or
1, not 1
is 1, not 1, what 2, matters 2
is 1, what 1, to 1, not 1, be 1, first 1
Shuffle/sort
be [1,1],1
first 1
is 1,1
matters [1,1]
not 1,1,1
or 1
to [1,1],1
what [1,1],1
Reduce
Reducer 1
Reducer 2
Collect output
be 3, first 1, is 2, matters 2, not 3, or 1, to 3, what 3

Example: counting (consecutive) pairs

- What are key-value pairs for input, intermediate and output?
- When possible, combine


Split
This is what it is.
It is not what it is not.
What it is not is this.
Map
Mapper 1
Mapper 2
Mapper 3
Combine
Shuffle/sort
Reduce
Reducer 1
Reducer 2
Collect output


Split
Map
This is what it is.
It is not what it is not.
What it is not is this.
Mapper 1
Mapper 2
Mapper 3
(this, is) 1, (is, what) 1
(what, it) 1, (it, is) 1
(it, is) 1, (is, not) 1 (not,what) 1,
(what, it) 1, (it, is) 1, (is, not) 1
(what, it) 1, (it, is) 1, (is, not) 1,
(not, is) 1, (is, this) 1
Shuffle/sort
(is, not) 1, (is, not) 1, (is, not) 1 (is, this) 1 (is,
what) 1, (it, is) 1, (it, is) 1, (it, is) 1, (it, is) 1
(not, is) 1, (not,what) 1, (this, is) 1,
(what, it) 1, (what, it) 1, (what, it) 1
Reduce
Reducer 1
Reducer 2
Collect output
(is, not) 3
(is, this) 1
(is, what) 1
(it, is) 4
(not, is) 1
(not,what) 1
(this, is) 1
(what, it) 3

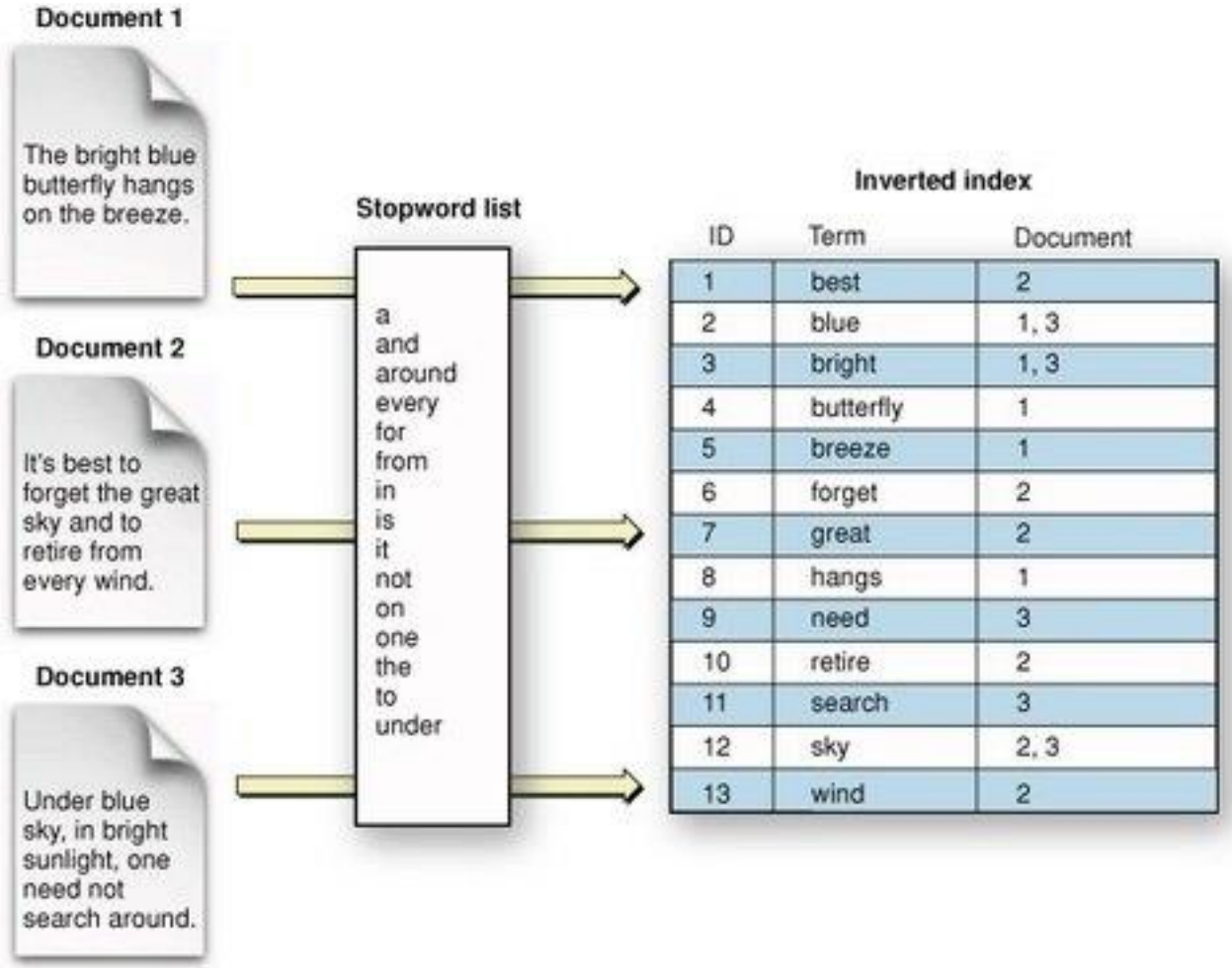

Split
Map
Combine
Reduce
Collect output
This is what it is.
It is not what it is not.
What it is not is this.
Mapper 1
Mapper 2
Mapper 3
(this, is) 1, (is, what) 1
(what, it) 1, (it, is) 1
(it, is) 1, (is, not) 1 (not,what) 1,
(what, it) 1, (it, is) 1, (is, not) 1
(what, it) 1, (it, is) 1, (is, not) 1,
(not, is) 1, (is, this) 1
(this, is) 1, (is, what) 1
(what, it) 1, (it, is) 1
(it, is) [1,1], (is, not) [1,1],
(not,what) 1, (what, it) 1
(what, it) 1, (it, is) 1, (is, not) 1,
(not, is) 1, (is, this) 1
Shuffle/sort
(is, not) [1,1], (is, not) 1 (is, this) 1 (is,
what) 1, (it, is) 1, (it, is) [1,1], (it, is) 1
(not, is) 1, (not,what) 1, (this, is) 1,
(what, it) 1, (what, it) 1, (what, it) 1
Reducer 1
Reducer 2
(is, not) 3
(is, this) 1
(is, what) 1
(it, is) 4
(not, is) 1
(not,what) 1
(this, is) 1
(what, it) 3

Focus for Today

- MR quick recap
  - Phases of Execution
  - Combiner optimisation
- Another MR example
- Hadoop Cluster
  - Key components
  - Deploying a Hadoop Cluster (Lab1)
- Run WordCount example (from example code)
  - Java
  - Python

Another example

- Inverted Index:



- What are…
  - …K_{in}, V_{in}?
  - …K_{inter}, V_{inter}?
  - …K_{out}, V_{out}?

Another example

- Inverted Index:
  - **Map** parses each document producing (word,documentID) pairs.
  - **Reduce** sorts for a given word all documentID's and produces a (word,list(documentID))
- Can add another MapReduce process to find top K words (based on document count)
- Used for:
  - Index construction for Google search (google)
  - Data mining (e.g. Facebook Lexicon)

Another example

- Inverted Index:

I have a list of documents (IDs) and content (text).

What are the MR phases to construct an Inverted Index?

Inverted Index

Input files

1.txt:
Cats hate apples and cats are cute

2.txt :
Batman is a good movie

3.txt:
Cats are cute and like treats

4.txt:
Batman is a good hero

5.txt:
Shopping is fun

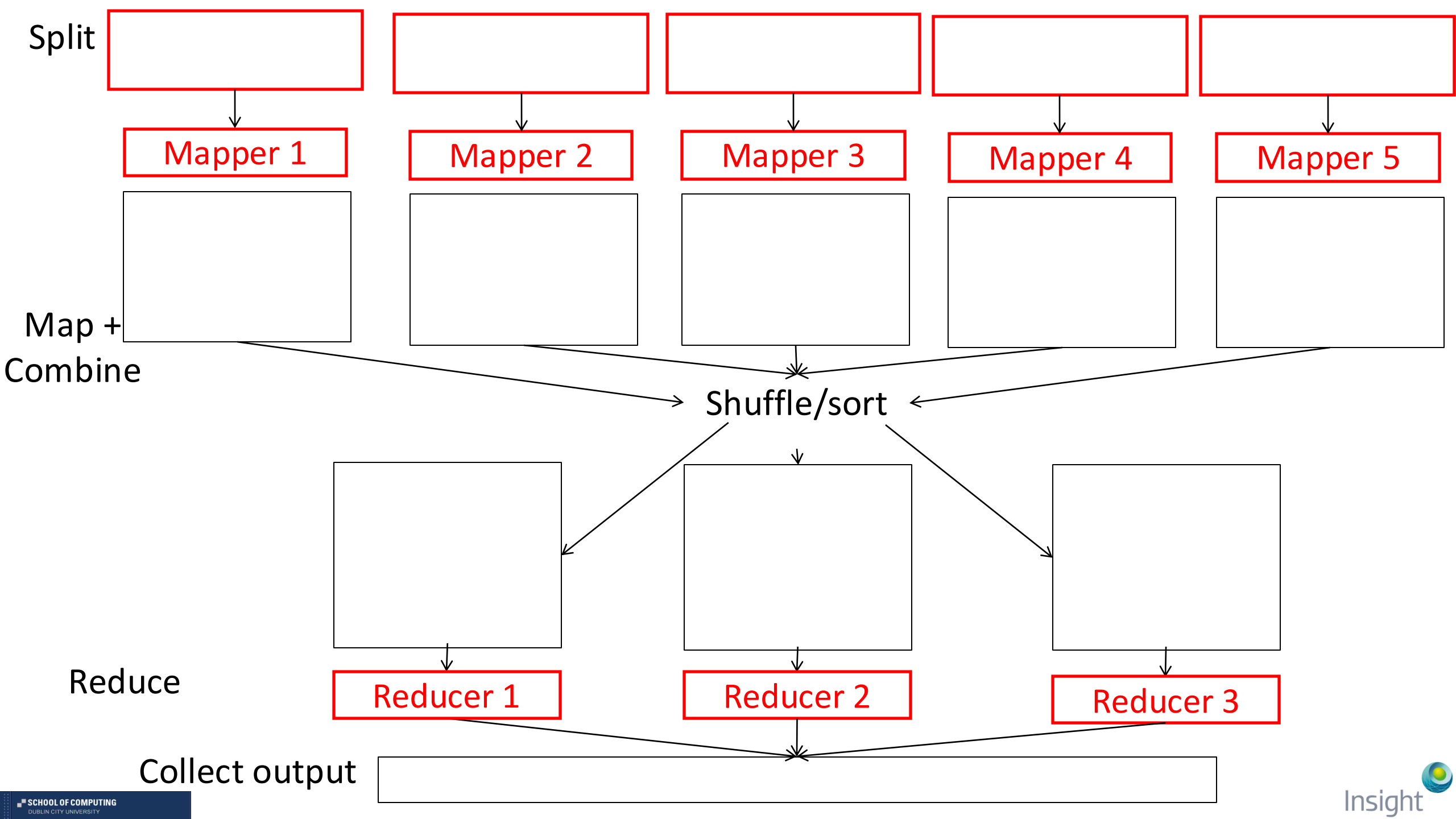
Notes:

Stopwords:
and, is, a, are

Mappers and reducers:
5 mappers and 3 reducers

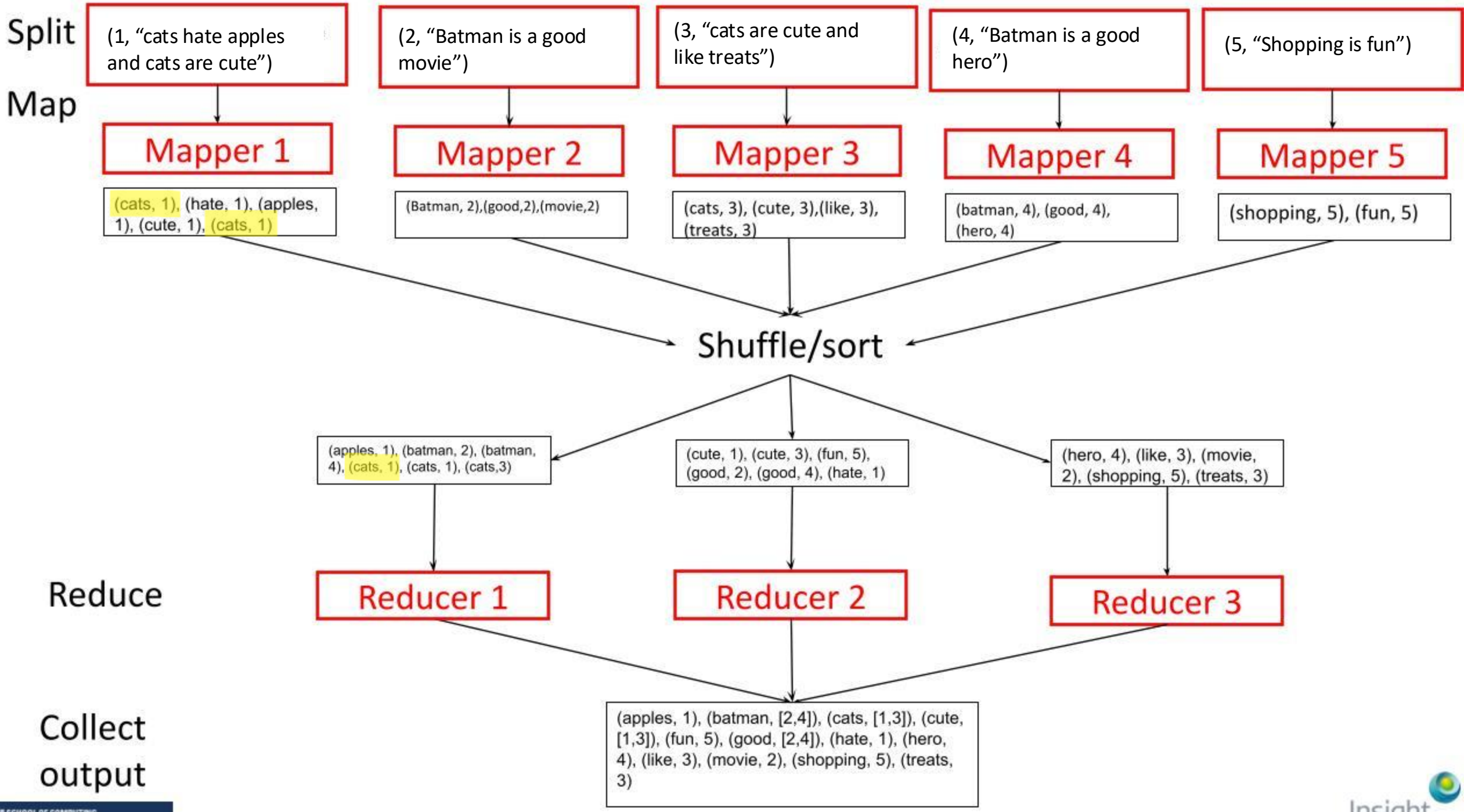
Optimisation:
Note that in this case, combiner optimisation can happen:

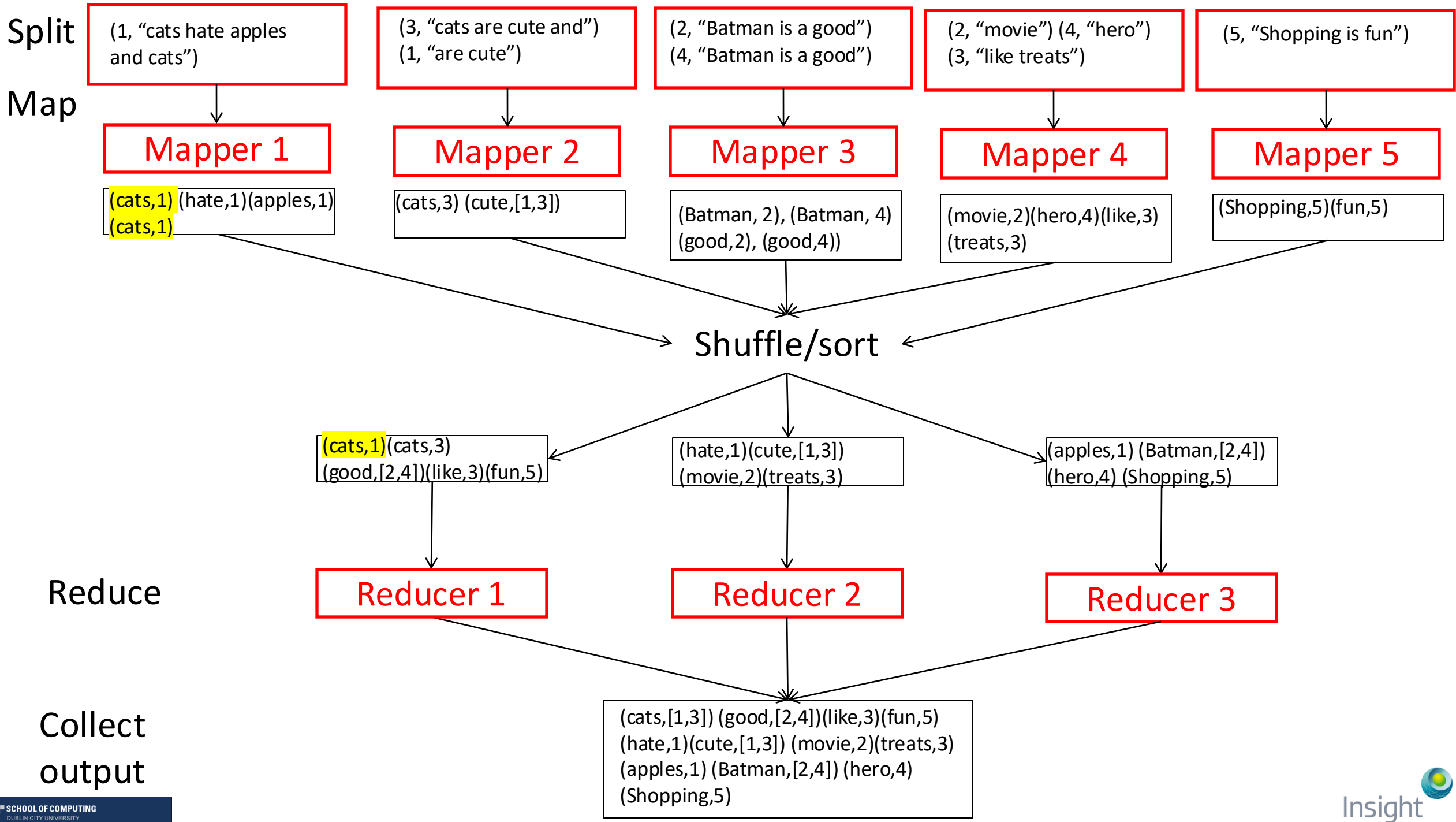
1. If you split sending multiple files to the same mapper, you can combine partial results by listing the word and all the files it appears in (from that mapper)

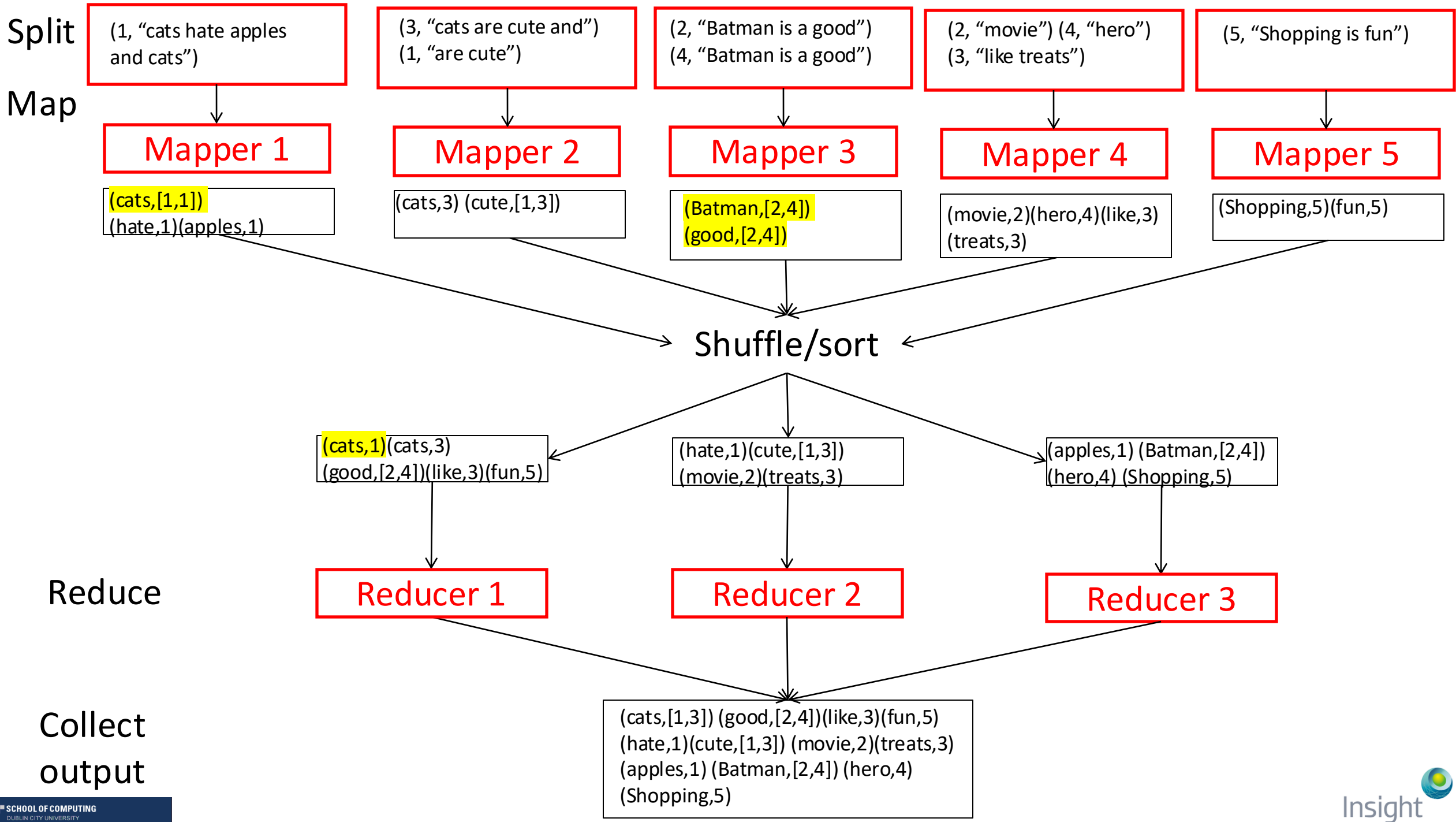

Split
Mapper 1
Mapper 2
Mapper 3
Mapper 4
Mapper 5
Map +
Combine
Shuffle/sort
Reduce
Reducer 1
Reducer 2
Reducer 3
Collect output

(Possible) Solutions

To be revealed in week 3


Split
Map
(1, “cats hate apples and cats are cute”)
(2, “Batman is a good movie”)
(3, “cats are cute and like treats”)
(4, “Batman is a good hero”)
(5, “Shopping is fun”)
Mapper 1
Mapper 2
Mapper 3
Mapper 4
Mapper 5
(cats, 1), (hate, 1), (apples, 1), (cute, 1), (cats, 1)
(Batman, 2),(good,2),(movie,2)
(cats, 3), (cute, 3),(like, 3), (treats, 3)
(batman, 4), (good, 4), (hero, 4)
(shopping, 5), (fun, 5)
Shuffle/sort
(apples, 1), (batman, 2), (batman, 4), (cats, 1), (cats, 1), (cats,3)
(cute, 1), (cute, 3), (fun, 5), (good, 2), (good, 4), (hate, 1)
(hero, 4), (like, 3), (movie, 2), (shopping, 5), (treats, 3)
Reduce
Reducer 1
Reducer 2
Reducer 3
Collect output
(apples, 1), (batman, [2,4]), (cats, [1,3]), (cute, [1,3]), (fun, 5), (good, [2,4]), (hate, 1), (hero, 4), (like, 3), (movie, 2), (shopping, 5), (treats, 3)


Split
Map
(1, “cats hate apples and cats”)
(3, “cats are cute and”) (1, “are cute”)
(2, “Batman is a good”) (4, “Batman is a good”)
(2, “movie”) (4, “hero”) (3, “like treats”)
(5, “Shopping is fun”)
Mapper 1
Mapper 2
Mapper 3
Mapper 4
Mapper 5
(cats,1) (hate,1)(apples,1) (cats,1)
(cats,3) (cute,[1,3])
(Batman, 2), (Batman, 4) (good,2), (good,4))
(movie,2)(hero,4)(like,3) (treats,3)
(Shopping,5)(fun,5)
Shuffle/sort
(cats,1)(cats,3) (good,[2,4])(like,3)(fun,5)
(hate,1)(cute,[1,3]) (movie,2)(treats,3)
(apples,1) (Batman,[2,4]) (hero,4) (Shopping,5)
Reduce
Reducer 1
Reducer 2
Reducer 3
Collect output
(cats,[1,3]) (good,[2,4])(like,3)(fun,5) (hate,1)(cute,[1,3]) (movie,2)(treats,3) (apples,1) (Batman,[2,4]) (hero,4) (Shopping,5)


Split
Map
(1, “cats hate apples and cats”)
(3, “cats are cute and”) (1, “are cute”)
(2, “Batman is a good”) (4, “Batman is a good”)
(2, “movie”) (4, “hero”) (3, “like treats”)
(5, “Shopping is fun”)
Mapper 1
Mapper 2
Mapper 3
Mapper 4
Mapper 5
(cats,[1,1]) (hate,1)(apples,1)
(cats,3) (cute,[1,3])
(Batman,[2,4]) (good,[2,4])
(movie,2)(hero,4)(like,3) (treats,3)
(Shopping,5)(fun,5)
Shuffle/sort
(cats,1)(cats,3) (good,[2,4])(like,3)(fun,5)
(hate,1)(cute,[1,3]) (movie,2)(treats,3)
(apples,1) (Batman,[2,4]) (hero,4) (Shopping,5)
Reduce
Reducer 1
Reducer 2
Reducer 3
Collect output
(cats,[1,3]) (good,[2,4])(like,3)(fun,5) (hate,1)(cute,[1,3]) (movie,2)(treats,3) (apples,1) (Batman,[2,4]) (hero,4) (Shopping,5)

Another example: Page Rank

- Page Rank basics:
  - Algorithm behind the quality of Google's search results
  - Measures the importance of web pages
  - Uses link structure instead of only content matching
- The WEB as a (directed) graph (with backward and forward links)
- PageRank for a page *P* is (intuitively) the sum over all backlinks of P of the PageRank of *v* linking to P (recursive!), divided by the number of forward-links of *v*.

Example: https://www.youtube.com/watch?v=P8Kt6Abq_rM

Focus for Today

- MR quick recap
  - Phases of Execution
  - Combiner optimisation
- Another MR example
- Hadoop Cluster
  - Key components
  - Deploying a Hadoop Cluster (Lab1)
- Run WordCount example (from example code)
  - Java
  - Python

Hadoop Cluster Key components

- HDFS
  - Hadoop distributed file systems
  - All commands on HDFS preceeded by `hdfs dfs`
  - All commands/arguments become arguments of `hdfs dfs`
- NameNode
  - Where everything is, but no data
- DataNode(s)
  - Store all data
  - Talk to task tracker (MR layer) an client app via NameNode

Lab1 – Hadoop MapReduce

Instructions:

https://csc1109.readthedocs.io/en/latest/lab1/